

SDV_Replica_POC

Daily Work Log - Full Command Reference

Author: Chittaranjan Baral (@SJLNS)

Date: 2026-08-08

Project: SDV_Replica_POC

Log version: v2.0 (detailed)

This log records **every command run today, in every shell used, in the exact order run**, with the working folder and the reason for each command - written so a fresher can reproduce the entire session from scratch, or diagnose why something on their own machine differs from this one. Reference sections for the general git workflow and the firmware build steps are included at the end for reuse in future sessions, independent of today's specific events.

1. COMMAND LOG - GIT BASH

Git Bash was the primary shell for this session because the project's build script (scripts/buildenv.sh) is a bash script and does not run in cmd or PowerShell.

Working folder: `/c/Git_source/SDV_Replica_POC`

```
$ git push -u origin main
remote: Repository not found. fatal: repository
'https://github.com/SJLNS/SDV_Replica_POC.git/' not found
```

Why: Publish the local repo's commits to the newly-added GitHub remote for the first time.

In plain terms: This failed because the repository did not exist on GitHub's server yet - 'git remote add' only stores an address locally, it does not create anything remotely. Fixed by creating the empty repository on github.com/new first, then re-running this same command.

```
$ git push -u origin main
Enumerating objects: 192, done. ... * [new branch] main -> main branch 'main' set up to
track 'origin/main'.
```

Why: Retry after creating the empty repository on GitHub.

```
$ git push origin v0.1.0
* [new tag] v0.1.0 -> v0.1.0
```

Why: Push the v0.1.0 annotated tag - tags are not included in a normal branch push and need their own explicit push.

```
$ git config --global credential.helper wincred
```

Why: GitHub Desktop's default credential helper (manager-core) crashed with a `MissingMethodException` on this laptop. Switched to wincred (Windows' native credential store) as a working alternative, applied globally (--global) since it's a laptop-wide git setting, not specific to this one repo.

```
$ git push origin main
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0 To
https://github.com/SJLNS/SDV_Replica_POC.git a2e5b86..89487f2 main -> main
```

Why: Retry the push now that wincred is the active credential helper.

```
$ git push origin v0.2.0
error: src refspec v0.2.0 does not match any error: failed to push some refs
```

Why: Attempt to push the v0.2.0 tag.

In plain terms: This failed because the v0.2.0 tag only existed in a different working copy - it was never created in THIS local repo. 'git push' can only push refs (branches/tags) that already exist locally; it does not invent them.

```
$ git tag -a v0.2.0 -m "Add buildenv.sh build wrapper for ZCU firmware targets"
```

Why: Create the missing tag locally, pointing at the current HEAD commit.

```
$ git tag
v0.1.0 v0.2.0
```

Why: Confirm the tag now exists locally.

```
$ git push origin v0.2.0
* [new tag] v0.2.0 -> v0.2.0
```

Why: Push the newly-created tag.

```
$ arm-none-eabi-gcc --version
bash: arm-none-eabi-gcc: command not found
```

Why: Check whether the ARM cross-compiler toolchain is installed and on PATH, before attempting any firmware build.

In plain terms: Toolchain was not yet installed on this laptop. Resolved by installing it via winget in PowerShell (see Section 2) and then reopening Git Bash so it would pick up the updated system PATH - an already-open terminal window does not see PATH changes made after it was opened.

```
$ arm-none-eabi-gcc --version
arm-none-eabi-gcc.exe (Arm GNU Toolchain 14.2.Rel1 (Build arm-14.52)) 14.2.1 20241119
```

Why: Re-check from a freshly-opened Git Bash window, after the winget install completed.

```
$ git status
HEAD detached at e1b9692 Changes not staged for commit: modified: pki/scripts/issue.sh
```

Why: Routine check of repo state before running the build - caught two issues before they became a problem.

```
$ git diff pki/scripts/issue.sh
old mode 100755 new mode 100644
```

Why: Inspect exactly what changed before deciding whether to keep or discard it - never discard an unexplained change without reading the diff first.

In plain terms: The diff showed only a file-permission change (executable bit removed), not a code change. This is a known Windows/NTFS behavior - it does not reliably preserve the Linux 'executable' flag when a file is copied. Safe to discard.

```
$ git config core.fileMode false
```

Why: Tell git to stop treating file-permission-only changes as modifications for this repo - prevents this same false-positive from appearing again on every future file.

```
$ git checkout -- pki/scripts/issue.sh
```

Why: Discard the (now-confirmed-harmless) mode-only change and restore the file exactly as it is in the last commit.

```
$ git status
HEAD detached at e1b9692 nothing to commit, working tree clean
```

Why: Confirm the working tree is clean.

```
$ git checkout main
```

Why: Fix the detached HEAD state by reattaching to the main branch - without this, any new commits made would risk becoming orphaned once switching branches later.

```
$ git status
On branch main nothing to commit, working tree clean
```

Why: Confirm back on main with a clean tree.

```
$ git log --oneline -5
89487f2 (HEAD -> main, tag: v0.2.0, origin/main) ... a2e5b86 (tag: v0.1.0) ... e1b9692
...
```

Why: Confirm commit history is intact and matches expectations after all the branch/detach activity.

```
$ ./scripts/buildenv.sh all clean
=== BUILD SUCCEEDED: zcu1-discovery === === BUILD SUCCEEDED: zcu2-nucleo ===
```

Why: Run the actual firmware build for both ZCU targets, from a clean state (clean = wipe any previous build-output/ first).

```
$ mkdir daily-logs
```

Why: Create the folder to hold daily work-log PDFs.

```
$ cat > daily-logs/README.md << 'EOF'
... (heredoc content) ...
EOF
```

Why: Create a README explaining the folder's naming convention and suggested content, using a bash heredoc to write multi-line content directly from the terminal without opening an editor.

```
$ git add daily-logs
warning: LF will be replaced by CRLF in daily-logs/README.md.
```

Why: Stage the new folder and its README for commit.

In plain terms: This warning is expected and harmless - git is normalizing line endings to Windows-style (CRLF) in the working directory copy while keeping Unix-style (LF) in the actual commit. It does not block the commit and is unrelated to the earlier file-mode issue.

```
$ git commit -m "chore: add daily-logs folder for work-log PDFs"
```

Why: Commit the staged folder.

```
$ git push origin main
```

Why: Publish the commit to GitHub.

2. COMMAND LOG - POWERSHELL

PowerShell was used specifically for installing the Arm GNU Toolchain via winget, since winget is a Windows-native tool most reliably driven from PowerShell (Administrator mode, per the screenshot title bar).

Working folder: **C:\Windows\system32** (default PowerShell start folder - not repo-specific)

```
PS> winget --version
v1.29.280
```

Why: Confirm the Windows Package Manager CLI is available before relying on it.

```
PS> [Environment]::GetEnvironmentVariable("Path", "User") | Out-File
$env:USERPROFILE\path_backup_before_arm_install.txt
```

Why: Save a copy of the current user PATH to a file BEFORE installing anything - precaution against a known bug where this specific installer has, in the past, replaced the entire PATH instead of adding to it.

```
PS> Get-Content $env:USERPROFILE\path_backup_before_arm_install.txt
```

Why: Confirm the backup file actually captured the PATH correctly before trusting it as a fallback.

```
PS> winget install Arm.GnuArmEmbeddedToolchain
Found GNU Arm Embedded Toolchain [Arm.GnuArmEmbeddedToolchain] Version 14.2.Rel1 Do
you agree to all the source agreements terms? [Y] Yes [N] No: Y Downloading ... 197 MB
Successfully installed
```

Why: Install the Arm GNU Toolchain (arm-none-eabi-gcc and related tools) system-wide.

```
PS> [Environment]::GetEnvironmentVariable("Path", "User")
C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2
rel1\bin;C:\Users\...\Python312\...\VS Code\bin;...Vector
Informatik...;C:\CTOOLS\bin;C:\CTOOLS\mingw32\bin
```

Why: Check the PATH immediately after install to see whether the known overwrite bug occurred.

```
PS> Get-Content $env:USERPROFILE\path_backup_before_arm_install.txt
(matched - Python, VS Code, WindowsApps, Vector, CTOOLS entries all still present)
```

Why: Compare against the pre-install backup line by line to confirm nothing was lost.

RESULT - No PATH damage

Every entry present before the install was still present after it, plus the new Arm toolchain entry. The known bug did not occur this time; the backup step meant this was confirmed rather than assumed.

3. COMMAND LOG - CMD (FAILED ATTEMPTS)

cmd (Command Prompt) was tried first, before switching to Git Bash for the remainder of the session, once it became clear git was not usable from cmd on this laptop.

Working folder: **C:\Git_source\SDV_Replica_POC**

```
C:\Git_source\SDV_Replica_POC>git remote add origin
git@github.com:/SDV_Replica_POC.git
The system cannot find the file specified.
```

Why: First attempt - copied a template command without replacing the placeholder.

In plain terms: was a placeholder meant to be replaced with the real GitHub username (SJLNS), not typed literally.

```
C:\Git_source\SDV_Replica_POC>git remote add origin  
https://github.com/SJLNS/SDV_Replica_POC.git  
'git' is not recognized as an internal or external command, operable program or batch  
file.
```

Why: Retry with the real username filled in.

In plain terms: Git for Windows was installed (confirmed via Start Menu search showing 'Git Bash'), but the 'git' command was not added to cmd's PATH during installation. Rather than fix cmd's PATH, the session switched to Git Bash instead, which has git working correctly out of the box - the simpler fix for the immediate goal of getting code pushed.

4. BUILD STEPS - REPOSITORY TO FIRMWARE BINARY (REFERENCE)

General-purpose reference: how to go from a fresh clone of the repository to a built firmware binary, any time, on any machine with the toolchain installed. Not specific to today's session - reusable for future sessions or a new team member's first build.

4.1 One-time setup (per machine)

```
$ git clone https://github.com/SJLNS/SDV_Replica_POC.git
$ cd SDV_Replica_POC
```

Why: Download the repository and enter its folder. Only needed once per machine.

```
$ arm-none-eabi-gcc --version
```

Why: Confirm the ARM cross-compiler toolchain is installed and on PATH. If this fails, install the Arm GNU Toolchain first (via winget on Windows, or apt on Linux - see the Execution Guide in docs/) before continuing.

4.2 Every build

```
$ cd SDV_Replica_POC # if not already there
$ ./scripts/buildenv.sh all clean
```

Why: Build both ZCU firmware targets from a clean state. Use 'zcu1-discovery' or 'zcu2-nucleo' instead of 'all' to build just one. Omit 'clean' to keep previous build-output/ artifacts and only rebuild what changed.

4.3 What buildenv.sh does internally, per target, in order

Step	Result
1. Preprocess main.c -> expands #include/#define, output: preprocessed/main.i	INFO
2. Compile main.c -> human-readable assembly, output: asm/main.s	INFO
3. Compile main.c -> machine object code, output: obj/main.o	INFO
4. Assemble startup.s -> machine object code, output: obj/startup.o	INFO
5. Link all .o files -> single executable, output: elf/elf + map/.map	INFO
6. Convert .elf -> raw binary for flashing, output: bin/.bin	INFO
7. Print a size report (text/data/bss) to confirm the binary fits the target's flash/RAM	INFO

In plain terms: Every one of these 7 steps is logged, in order, with a timestamp, into a file under build-output/logs/. If ANY step fails, the script stops immediately (it does not continue to later steps with broken input) and prints a large red banner naming exactly which step failed and which log file has the full compiler error text.

5. GIT WORKFLOW REFERENCE (CHEAT SHEET)

General reference for common git operations on this repo, independent of today's session. Run from Git Bash, from inside the repo folder, unless noted otherwise. Rows marked [CAUTION] or [DANGER] can lose uncommitted work if used carelessly - read the note before running.

5.1 Check-in (saving and publishing work)

Action	Command	What it does
Check status	<code>git status</code>	Shows what's changed, staged, or untracked. Run this often.
Stage a file	<code>git add path/to/file</code>	Marks a specific file's changes to be included in the next commit.
Stage everything	<code>git add -A</code>	Stages all changes (new, modified, deleted files) repo-wide.
Commit	<code>git commit -m "message"</code>	Saves staged changes as a new snapshot in local history. Write a clear message.
Push	<code>git push origin main</code>	Uploads local commits on the current branch to GitHub.
Push a tag	<code>git push origin v1.0.0</code>	Tags are not included in a normal push - must be pushed explicitly.

5.2 Check-out (retrieving / switching)

Action	Command	What it does
Pull latest	<code>git pull origin main</code>	Downloads and merges the latest commits from GitHub into your current branch.
Fetch only	<code>git fetch origin</code>	Downloads latest info from GitHub WITHOUT merging - safe way to look before pulling.
Switch branch	<code>git checkout main</code>	Moves your working folder to reflect a different branch's latest commit.
Checkout one file from history [CAUTION]	<code>git checkout -- path/to/file</code>	Restores a single file to how it looked at a specific past commit, without touching anything else.

5.3 Branching and merging

Action	Command	What it does
List branches	<code>git branch</code>	Shows all local branches; current one is marked with *.
Create + switch to new branch	<code>git checkout -b feature-name</code>	Creates a new branch off the current commit and switches to it immediately.

Switch branch (modern)	<code>git switch feature-name</code>	Newer, clearer alternative to 'git checkout' for just switching branches.
Merge a branch in [CAUTION]	<code>git merge feature-name</code>	Run FROM the branch you want to merge INTO (e.g. main). Combines feature-name's changes into the current branch.
Delete a local branch [CAUTION]	<code>git branch -d feature-name</code>	Deletes a branch that's already merged. Use -D (capital) to force-delete an unmerged one - be sure first.

5.4 Discarding changes

Action	Command	What it does
Discard changes to one file (not yet staged) [CAUTION]	<code>git checkout -- path/to/file</code>	Restores that file to its last-committed state. Uncommitted edits to it are permanently lost.
Discard changes to one file (modern) [CAUTION]	<code>git restore path/to/file</code>	Same effect as above, newer/clearer command name.
Unstage a file (keep the edits)	<code>git restore --staged path/to/file</code>	Removes a file from the staging area without touching its actual content changes.
Discard ALL uncommitted changes, repo-wide [DANGER]	<code>git reset --hard HEAD</code>	Throws away every uncommitted change in the entire working tree. Cannot be undone. Double-check git status first.
Remove untracked files too [DANGER]	<code>git clean -fd</code>	Deletes files git doesn't know about at all (e.g. stray build output). Cannot be undone - review with 'git clean -n' (dry run) first.

5.5 Rebasing and history

Action	Command	What it does
Rebase current branch onto another [CAUTION]	<code>git rebase main</code>	Replays your branch's commits on top of main's latest state, for a cleaner linear history. Can cause conflicts to resolve manually.
Abort a rebase in progress	<code>git rebase --abort</code>	Cancel an in-progress rebase and returns to the exact state before it started - safe escape hatch if a rebase goes wrong.
Amend the last commit [CAUTION]	<code>git commit --amend</code>	Changes the most recent commit's message or content instead of creating a new one. Do NOT use this on a commit already pushed and shared.

View history	<code>git log --oneline --decorate</code>	Compact, one-line-per-commit view with branch/tag labels - good for a quick sanity check.
--------------	---	---

6. ISSUES FACED - SUMMARY TABLE

Step	Result
Pushed before creating the GitHub repo -> 'repository not found'	FAIL
developer.arm.com toolchain page deprecated, redirects to a confusing GitLab layout	FAIL
GitLab Releases page empty for gnu-toolchains-for-arm project	FAIL
cmd: placeholder username typed literally	FAIL
cmd: git not on PATH at all	FAIL
GitHub Desktop's manager-core credential helper crashed (MissingMethodException)	FAIL
Pushed a tag that only existed in a different local copy, not this one	FAIL
Detached HEAD state after some earlier checkout activity	FAIL
False-positive 'modified' file from Windows/NTFS file-mode handling	FAIL
winget install PATH-overwrite risk (known bug, checked, did not occur)	OK
All ZCU firmware builds - sandbox and laptop	OK

7. OPEN ITEMS FOR NEXT SESSION

- STM32 Discovery/Nucleo exact part numbers not yet confirmed (Ethernet PHY question still open).
- Generic linker script (STM32_GENERIC.ld) and placeholder vector table not yet matched to real part-number datasheet values - must fix before flashing real hardware.
- RPi5 Stage 1 (stock Raspberry Pi OS flash + validation) not yet started.
- QNX SDP release with confirmed RPi5 BSP support not yet identified.
- Optional: proper Git Credential Manager reinstall to fix the manager-core crash (not urgent - wincred works fine as the alternative).

8. GLOSSARY (FOR NEW READERS)

PATH	The list of folders Windows/Linux searches to find a program when you type its name without a full file path.
Git Bash	A terminal that runs Linux-style bash commands on Windows. Different from cmd or PowerShell - buildenv.sh only runs here.
Credential helper	The tool Git uses to remember your GitHub login between pushes (wincred, manager-core, store, cache are common options on Windows).
Personal Access Token (PAT)	A token GitHub issues instead of your account password, used for git command-line operations.

Detached HEAD	Your repo is looking at one fixed commit instead of following a branch forward. Fixed with 'git checkout main'.
Git tag	A permanent label on one specific commit, usually marking a release.
File mode / executable bit	A Linux permission flag marking a file as runnable; Windows/NTFS doesn't preserve it reliably on copy.
Toolchain	The compiler + assembler + linker set needed to turn source code into a binary the target chip can run.
Staging area	Where 'git add' puts changes before they're included in a commit - lets you commit only part of what you've changed.
Rebase	Rewrites a branch's commits on top of a different starting point, for a cleaner linear history, instead of merging.

End of log - 2026-08-08. Repo: github.com/SJLNS/SDV_Replica_POC